Linear Models 3: Non-linearity Lab

Dr. Luisa Barbanti and Dr. Matteo Tanadini

Applied Machine Learning and Predictive Modelling 1, HS25 (HSLU)

Contents

1 Getting data

We are going to use the *d.trees* data set again. This data set contains the growth rates of 557 trees.

```
d.trees <- read.csv2("../../Datasets/TreesChamagne2017_Lab_modified.csv",
                     stringsAsFactors = TRUE)
##
str(d.trees)
```

```
'data.frame':   557 obs. of  12 variables:
 $ growth.rate       : num  0.702 1.139 1.394 1 1.355 ...
 $ species           : Factor w/ 4 levels "Beech","Larch",..: 1 1 1 4 4 4 1 1 1 1 ...
 $ site              : int  1 1 12 12 12 12 12 12 12 12 ...
 $ Density.tree.Class: Factor w/ 3 levels "high","low","medium": 1 2 3 3 1 1 2 3 2 3 ...
 $ age               : num  55 42 60 63 58 57 65 61 65 58 ...
 $ size              : num  23296 33718 57300 67297 97764 ...
 $ density.site      : num  43.6 43.6 35.8 35.8 35.8 ...
 $ density.tree      : num  6.03 3.91 5.54 5.06 5.89 ...
 $ diversity.tree    : num  1 1.14 1.98 2.67 2.19 ...
 $ diversity.site    : num  1.28 1.28 2.27 2.27 2.27 ...
 $ sp.richness       : int  1 1 2 2 2 2 2 2 2 2 ...
 $ SiteID            : int  1 1 12 12 12 12 12 12 12 12 ...
```

```
head(d.trees)
```

```
  growth.rate species site Density.tree.Class age  size density.site
1         0.7   Beech    1               high  55 23296           44
2         1.1   Beech    1                low  42 33718           44
3         1.4   Beech   12             medium  60 57300           36
4         1.0  Spruce   12             medium  63 67297           36
5         1.4  Spruce   12               high  58 97764           36
6         1.2  Spruce   12               high  57 67373           36
  density.tree diversity.tree diversity.site sp.richness SiteID
1          6.0            1.0            1.3           1      1
2          3.9            1.1            1.3           1      1
3          5.5            2.0            2.3           2     12
4          5.1            2.7            2.3           2     12
5          5.9            2.2            2.3           2     12
6         10.2            2.0            2.3           2     12
```

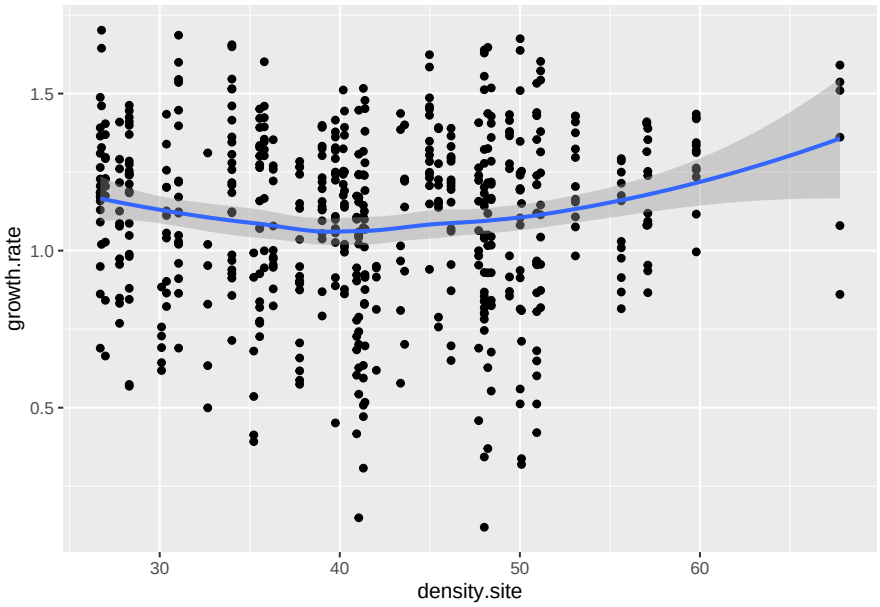
2 Polynomials

2.1 Graphical analysis

In the previous lecture we fitted a model on tree growth that assumed all the effects of continuous predictors to be linear. This may not always be true. For example, the graph below indicated that the effect of *density.site* may be non-linear.

```
library(ggplot2)
gg.density.site <- ggplot(data = d.trees,
       mapping = aes(y = growth.rate,
```

```
                    x = density.site)) +
  geom_point()
##
gg.density.site +
  geom_smooth()
```



We may want to model the non-linearity shown by the smoother with a quadratic effect.

2.2 Quadratic effects

We now try to relax the assumption of linearity for *density.site* by adding a quadratic term to the model. For the sake of simplicity, we fit a model that does not contain interactions.

```
## 1) model with a linear effect for density.site
lm.trees.1 <- lm(growth.rate ~ species + age + diversity.site +
                   density.site,
                 data = d.trees)
##
## 2) model with a quadratic effect for density.site
lm.trees.2 <- update(lm.trees.1, . ~ . + I(density.site^2))
```

Note that we use the `I()` function to make clear that we want an quadratic term, rather than a two-fold interaction that can also be specified with `^2`. See the appendix for further information.

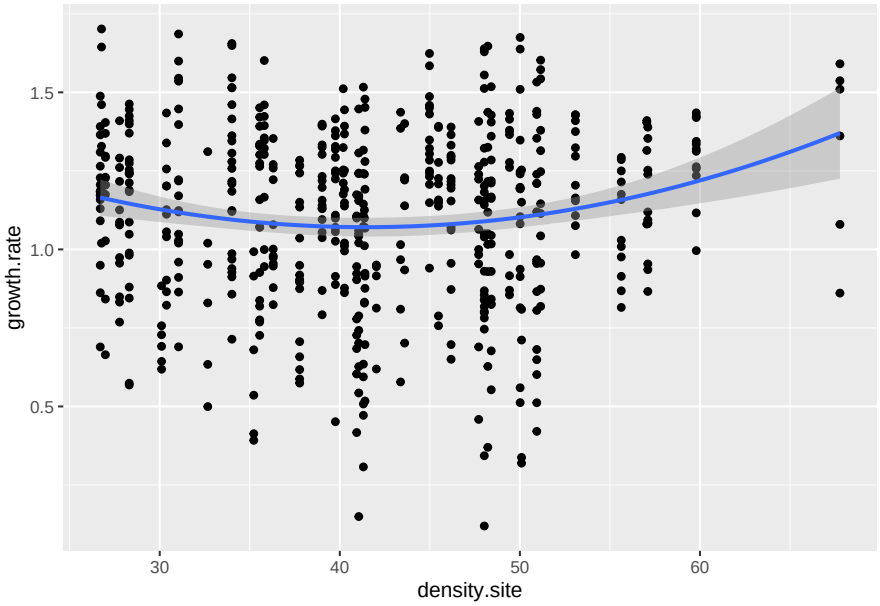
We test this quadratic term with a F-test.

```
anova(lm.trees.1, lm.trees.2)
```

```
Analysis of Variance Table

Model 1: growth.rate ~ species + age + diversity.site + density.site
```

```
Model 2: growth.rate ~ species + age + diversity.site + density.site +
    I(density.site^2)
  Res.Df  RSS Df Sum of Sq    F  Pr(>F)
1    550 35.7
2    549 34.7  1     0.982 15.5 9.3e-05 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

There is strong evidence that *density.site* needs a quadratic term. This confirms the plot we have seen above, where the effect of "tree density" appeared to be non-linear. Let's try to visualise the effect of density as modelled here. Note that the blue line of the graph below is not a smoother, but rather a quadratic polynomial.

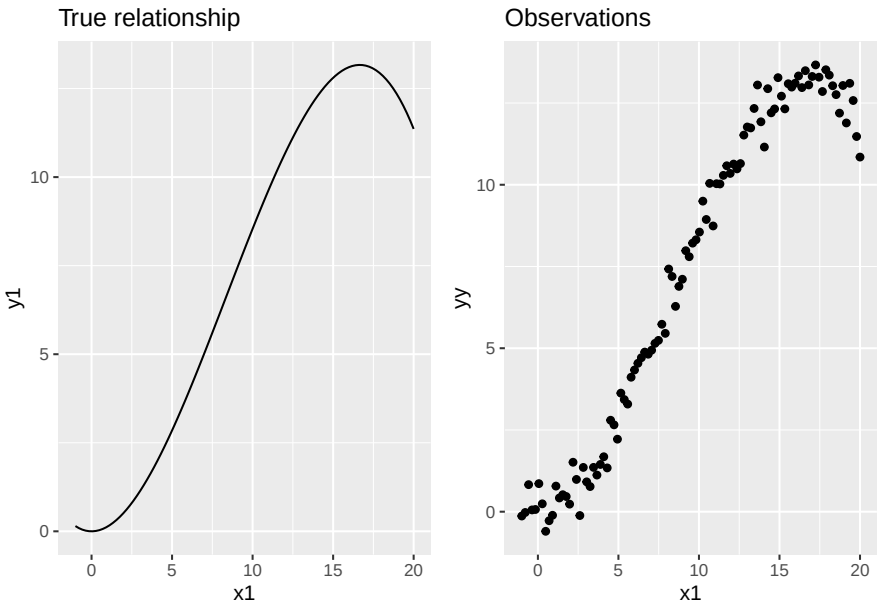


Note: Sometimes quadratic terms can be highly collinear with the corresponding linear term (this concept is explained in the Appendix). To avoid collinearity problems when adding quadratic terms, we can use the `poly()` function. This function creates orthogonal polynomials that are not affected by collinearity issues.

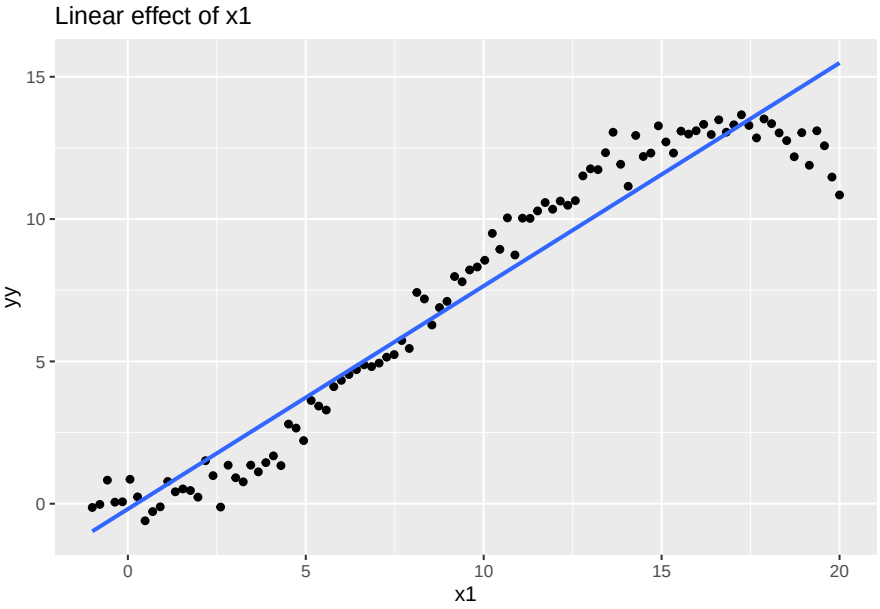
```
lm.trees.3 <- lm(growth.rate ~ species + age + diversity.site +
                           poly(density.site, degree = 2),
                 data = d.trees)
```

2.3 More complex non-linear relationships

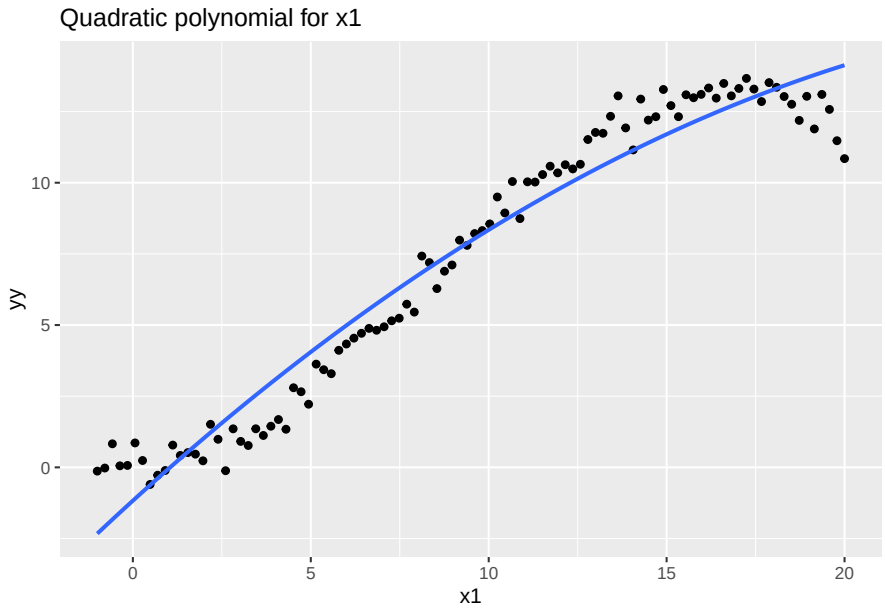
Let's turn our attention to a simulated example.



If we were to assume a linear relationship, we would be locally under- and overestimating the true expected value. In other words, our model would not be a good simplified representation of reality.

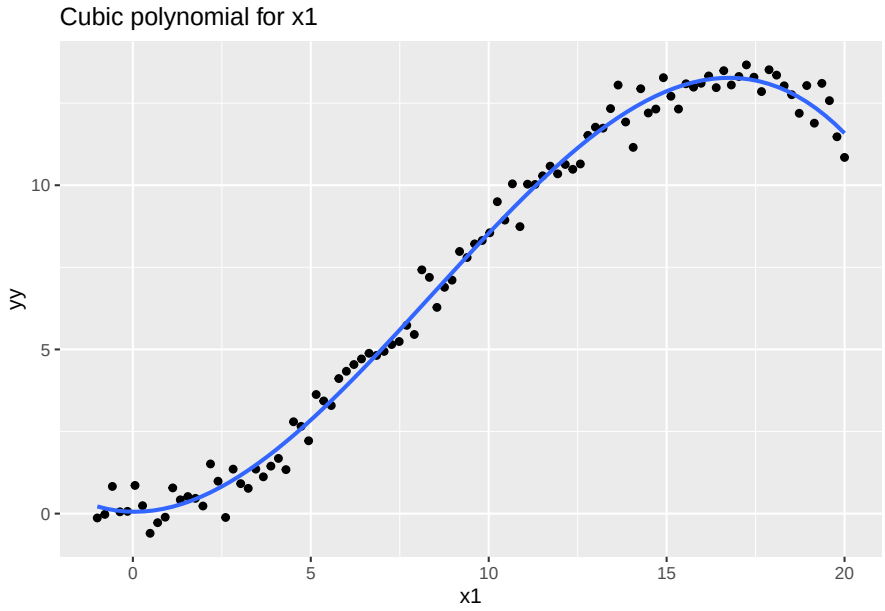


Let's try to model this with a quadratic term.



The quadratic term is not enough to model the relationship between `x1` and `yy`. More flexibility is required.

Let's try higher order polynomials (e.g. a cubic polynomial)



The regression line fits very well the data. As a matter of fact, the true relationship was simulated from a cubic polynomial.

So in this case, we could model these data with the following linear model.

```
lm.cubic <- lm(yy ~ poly(x1, degree = 3), data = sim.data)
summary(lm.cubic)
```

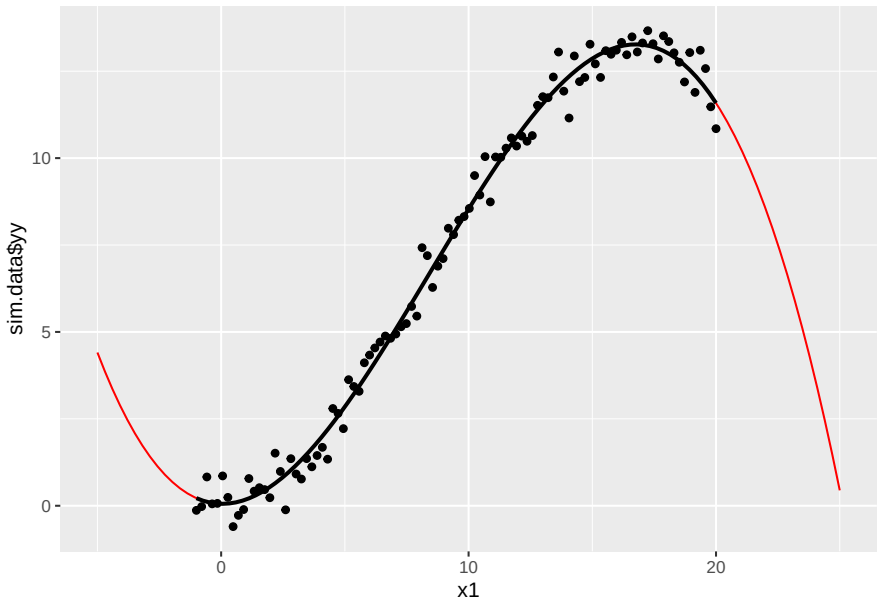
```
Call:
lm(formula = yy ~ poly(x1, degree = 3), data = sim.data)

Residuals:
    Min      1Q  Median      3Q     Max
-1.1966 -0.2999 -0.0124  0.3347  1.0944

Coefficients:
                      Estimate Std. Error t value Pr(>|t|)
(Intercept)             7.2580     0.0459   158.0   <2e-16 ***
poly(x1, degree = 3)1  47.9821     0.4595   104.4   <2e-16 ***
poly(x1, degree = 3)2  -6.2530     0.4595   -13.6   <2e-16 ***
poly(x1, degree = 3)3 -10.2169     0.4595   -22.2   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.46 on 96 degrees of freedom
Multiple R-squared:  0.992, Adjusted R-squared:  0.992
F-statistic: 3.86e+03 on 3 and 96 DF,  p-value: <2e-16
```

2.4 Are polynomials the ultimate solution for modelling non-linear relationships?

By looking at the examples above, it may appear that polynomials can solve any problem of non-linear relationships. Unfortunately, this is not the case. Indeed, polynomials, although very useful in very many situations, suffer from the following shortcomings:

- polynomials can be highly collinear and lead to testing or even estimation issues. Fortunately, orthogonal polynomials (i.e. the `poly()` function) can solve this problem.
- they are not reliable outside the fitting region (see graph below).
- there is no simple rule that tells us what degree to use. In these examples, lower order polynomials were "good enough". In reality, however, higher orders may be needed.

Predictions for regions outside the fitting region (i.e. where no data are present) are referred to as "extrapolation". Extrapolation is often problematic when polynomials are used.



In general, the higher the degree of the fitted polynomial, the less reliable the extrapolation becomes.

3 Regression splines

Regression splines have better properties than polynomials in terms of reliability outside the fitting region and do not suffer from collinearity issues.

Regression splines are based on basis functions. There are many different types of basis functions that can be used to build a regression spline.

As for polynomials, the dimension (resp. the degree) of the regression spline must be set by the user.

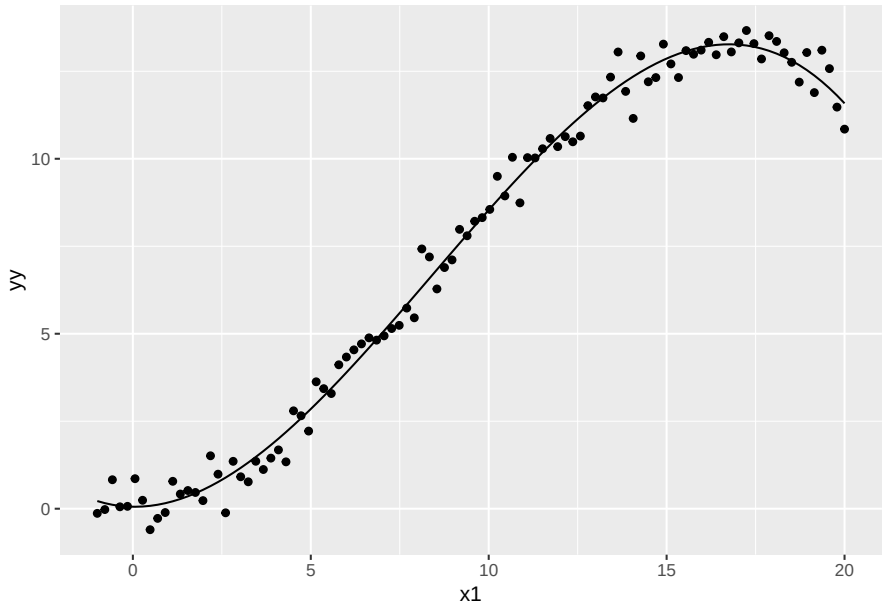
Let's look at an example of regression splines. Note that we use B-splines here.

```
library(splines)
lm.regressionSplines <- lm(yy ~ bs(x1, df = 3), data = sim.data)
summary(lm.regressionSplines)
```

```
Call:
lm(formula = yy ~ bs(x1, df = 3), data = sim.data)

Residuals:
    Min      1Q  Median      3Q     Max
-1.1966 -0.2999 -0.0124  0.3347  1.0944

Coefficients:
               Estimate Std. Error t value Pr(>|t|)
(Intercept)       0.219      0.177    1.24     0.22
bs(x1, df = 3)1  -2.219      0.514   -4.32  3.8e-05 ***
bs(x1, df = 3)2  19.067      0.331   57.59  < 2e-16 ***
bs(x1, df = 3)3  11.365      0.278   40.89  < 2e-16 ***
---
```

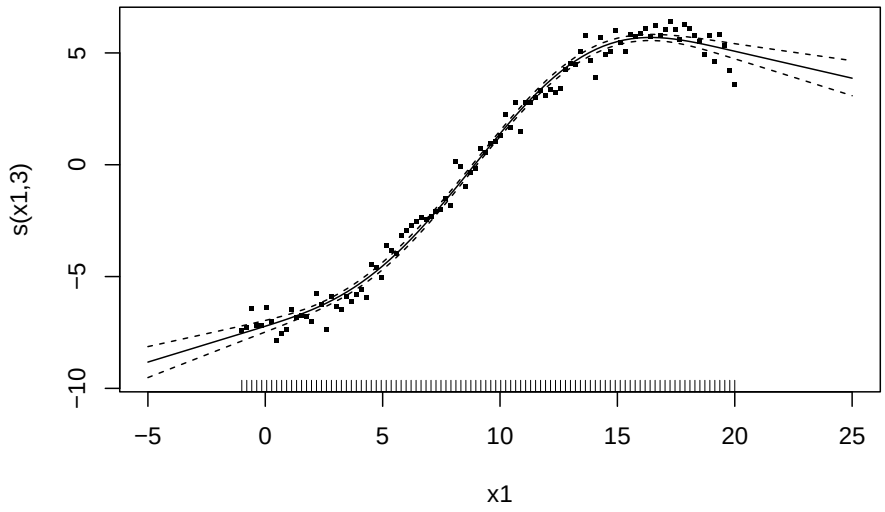
```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.46 on 96 degrees of freedom
Multiple R-squared:  0.992, Adjusted R-squared:  0.992
F-statistic: 3.86e+03 on 3 and 96 DF,  p-value: <2e-16
```

Let's look at the fit of this model.



As expected the fit is almost identical to the one obtained with the cubic polynomial.

Let's now look at the predictions for regression splines outside the fitting region[1].



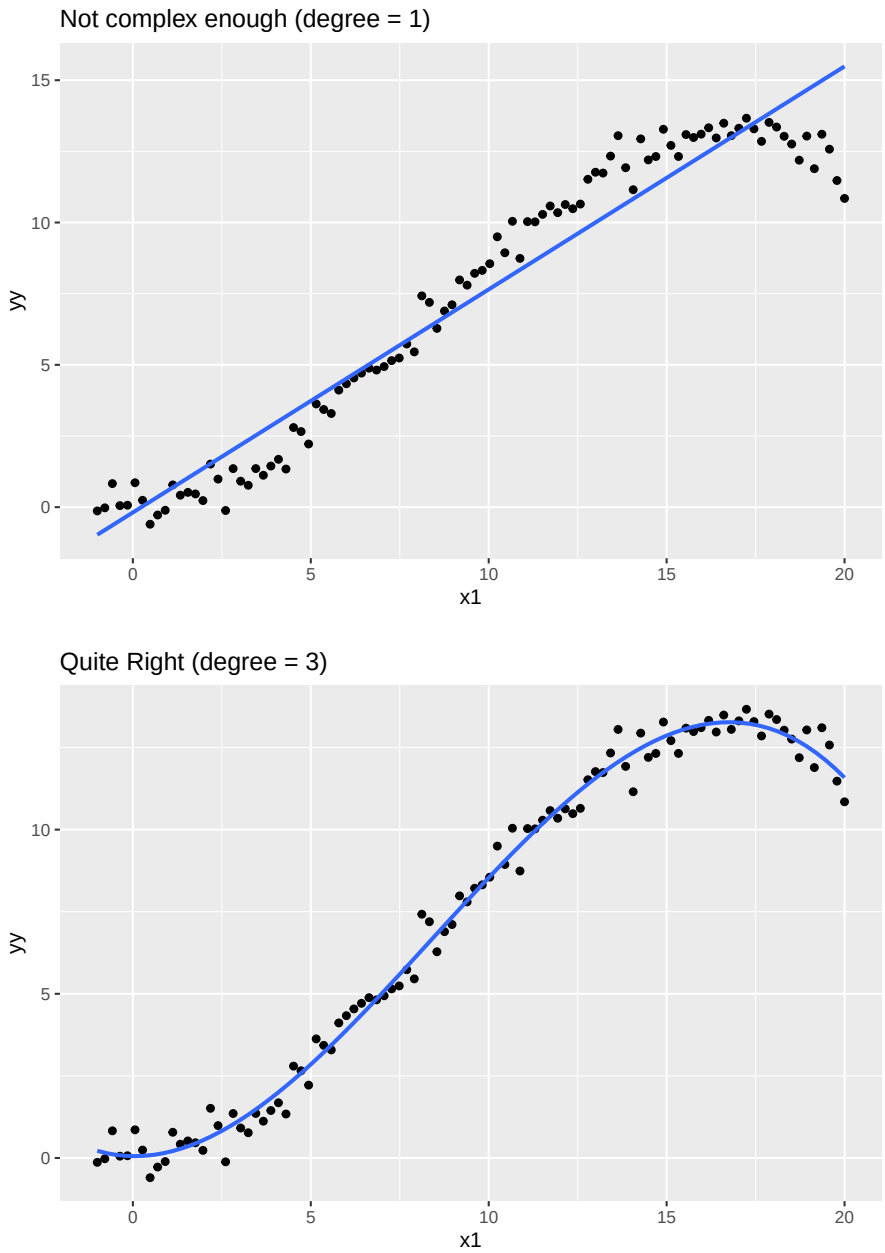
The behaviour outside the fitting region is more appropriate and reliable than for the cubic polynomial model. Indeed, splines just use linear extrapolation for regions where no data is available.

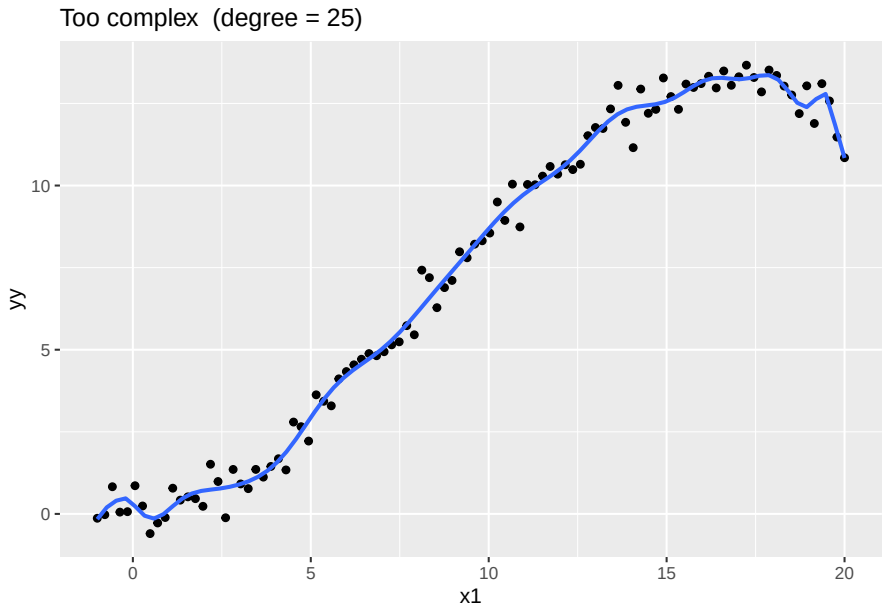
[1](***) Note that the graph shown below is not exactly the one obtained for a B-spline. For the sake of simplicity, this graph is obtained by using an unpenalised thin regression spline. These details are far beyond the level required for this course and can be safely ignored.

3.1 Degree of complexity, how much is enough?

As for polynomials, the user must define the complexity of the regression splines. This is done through the definition of the basis dimension. In the example above, the polynomial had order three and the B-spline had dimension three as well.

In most practical situations it is not clear what the right degree complexity is. The following graph shows three different situations.





4 Smoothing splines

A valid solution to the problem of choosing the correct degree of complexity to model non-linear relationships is to use **smoothing splines**.

The main difference between regression and smoothing splines is that the latter are penalised. In other words, **the smoothing spline is "automatically" choosing the appropriate degree of complexity**.

Smoothing splines can deal with just one predictor. However, in real settings, we want to fit models that contain several predictors. Generalised Additive Models are the extension to smoothing splines that enables the user to fit models that contain several predictors simultaneously.

5 Generalised Additive Models

The currently most powerful implementation of Generalised Additive Models, GAMs for short, is to be found in the {*mgcv*} package[2].

Let's fit this model with the `gam()` function found in the {*mgcv*} package.

```
library(mgcv)
gam.1 <- gam(yy ~ s(x1), data = sim.data)
summary(gam.1)
```

```
Family: gaussian
Link function: identity

Formula:
yy ~ s(x1)
```

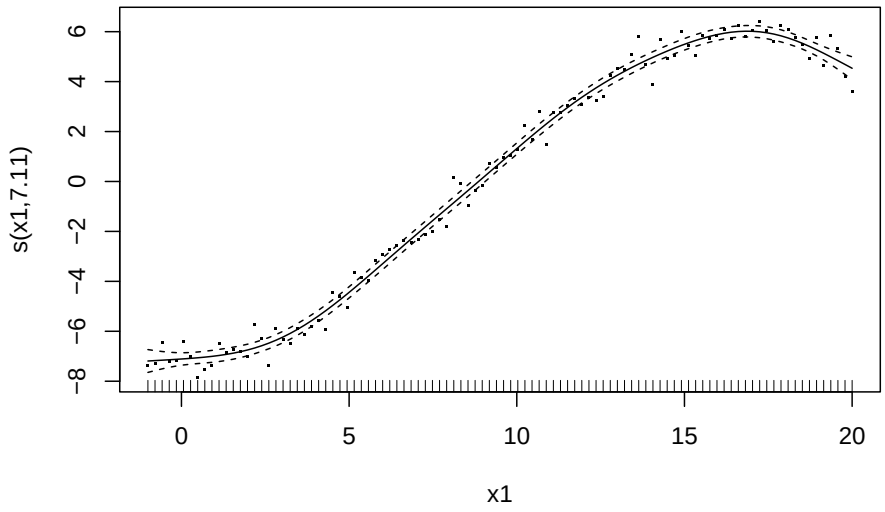
[2]The {*gam*} package present in the base **R** installation also implements GAMs (see its `gam()` function). However, GAMs in the {*gam*} package are implemented based on the backfitting algorithm, which has been superseded. In addition, the {*mgcv*} package implements a much wider choice of methods and tools.

```
Parametric coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   7.2580     0.0463     157   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Approximate significance of smooth terms:
       edf Ref.df    F p-value
s(x1) 7.11   8.16 1397  <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

R-sq.(adj) =  0.991   Deviance explained = 99.2%
GCV = 0.23338  Scale est. = 0.21446   n = 100
```

The `s()` function allows the user to specify smooth terms. In this case, the effect of *x1*, the unique predictor present in the model, is modelled with a smooth term (in this case, this very simple GAM is fully equivalent to a smoothing spline).

Let's visualise the estimated effect of *x1*.

```
plot(gam.1, residuals = TRUE, cex = 2)
```



5.1 Tree growth example

In reality, we often fit models that contain several predictors. Let's fit a GAM model to the tree growth dataset. This model contains the factor *species* and two continuous predictors. Both of them are allowed to have a non-linear, smooth, effect.

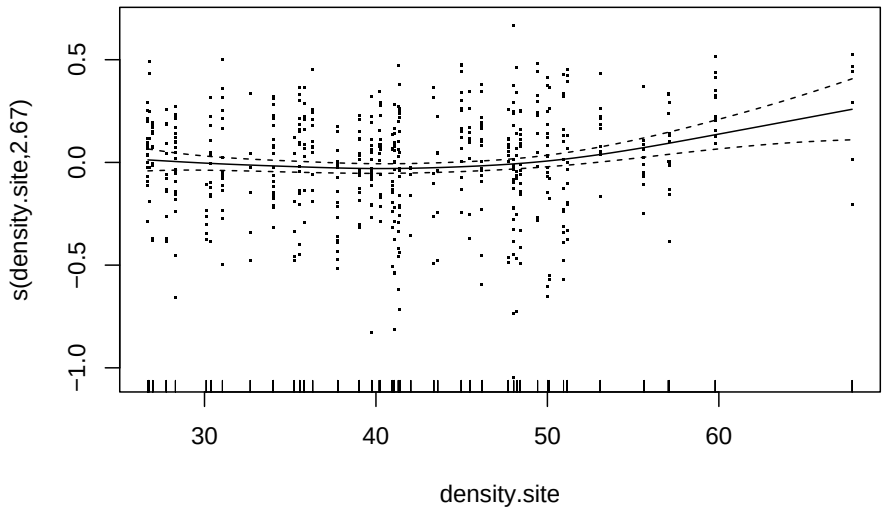
```
gam.trees.1 <- gam(growth.rate ~ species +
                     s(density.site) + s(diversity.site),
                   data = d.trees)
summary(gam.trees.1)
```

```
Family: gaussian
Link function: identity

Formula:
growth.rate ~ species + s(density.site) + s(diversity.site)

Parametric coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept)     1.2522     0.0213   58.80  < 2e-16 ***
speciesLarch   -0.3015     0.0306   -9.84  < 2e-16 ***
speciesOak     -0.1863     0.0302   -6.18  1.3e-09 ***
speciesSpruce  -0.0980     0.0311   -3.16   0.0017 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Approximate significance of smooth terms:
                   edf Ref.df  F p-value
s(density.site)   2.67   3.36  5 0.00143 **
s(diversity.site) 1.00   1.00 15 0.00012 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

R-sq.(adj) =  0.196   Deviance explained = 20.5%
GCV = 0.064137  Scale est. = 0.063253  n = 557
```

The predictor *species* is dealt with exactly the same way as in linear models[3]. Indeed, the summary output for this predictor has the same structure the one obtained when fitting a linear model.

The summary output indicates that there there is strong evidence that *density.site* has non-linear effect on the response variable. The *estimated degrees of freedom*, `edf` in the output, quantify the complexity of the smooth function. For the predictor *density.site* the `edf` is about 2.7. Let's visualise the effect of this variable.

```
plot(gam.trees.1, residuals = TRUE, select = 1)
```



[3]Indeed, it would make no sense to try to fit a non-linear effect for a variable that is categorical. It goes without saying that smooth terms can only be used for continuous variables.

Not unexpectedly, this effect is very similar to a quadratic effect. Indeed, the `edf` value was 2.7.

Let's turn our attention to the other smooth term in the model. The estimated degrees of freedom for *diversity.site* equal one. This means that this term is considered to be a simple linear term. In other words, there is no evidence that *diversity.site* has a non-linear effect on the response variable. Nevertheless, there is strong evidence (see the corresponding p-value) that the effect of this predictor is not zero.

As this predictor has a linear effect, we may want to refit the model accordingly.

```
gam.trees.2 <- gam(growth.rate ~ species +
                     s(density.site) + diversity.site,
                   data = d.trees)
summary(gam.trees.2)
```

```
Family: gaussian
Link function: identity

Formula:
growth.rate ~ species + s(density.site) + diversity.site

Parametric coefficients:
               Estimate Std. Error t value Pr(>|t|)
(Intercept)      1.1041     0.0435   25.37  < 2e-16 ***
speciesLarch    -0.3015     0.0306   -9.84  < 2e-16 ***
speciesOak      -0.1863     0.0302   -6.18  1.3e-09 ***
speciesSpruce   -0.0980     0.0311   -3.16  0.00169 **
diversity.site   0.0581     0.0150    3.87  0.00012 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Approximate significance of smooth terms:
                 edf Ref.df F p-value
s(density.site) 2.67   3.36 5  0.0014 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

R-sq.(adj) =  0.196   Deviance explained = 20.5%
GCV = 0.064137  Scale est. = 0.063253  n = 557
```

The results do not change as the two models (i.e. `gam.trees.1` and `gam.trees.2`) are fully equivalent.

6 Summary for data scientists

Polynomials:

- can be used to model non-linear effects
- in practice, only low-degree polynomials are used (quadratic or cubic)
- can be affected by collinearity, therefore, orthogonal polynomials are used
- cannot be used to make predictions outside the fitting region (extrapolation)

Regression splines:

- have better properties than polynomials
- but in reality, you almost never use them
- they were introduced here for didactic reasons as they are component of smoothing splines

Smoothing splines:

- have better properties than regression splines
- but in reality, you almost never use them
- they were introduced here for didactic reasons as they are a building block of Generalised Additive Models (GAMs)

Generalised Additive Models:

- are often used to model non-linear effects
- they are an extension of the linear model (and are thus still a parametric method)

Shall we use polynomials or GAMs?

- quadratic or cubic polynomials are more intuitive to understand and use
- however, when using them we arbitrarily decide how complex is the relationship. GAMS have a more objective use as their complexity is computed from the data
- in reality, quadratic and cubic polynomials are not enough to model complex relationships
- depending on the data, the situation, the audience and your skills you may choose one or the other approach[4].

7 Exercise

Get the "Girth" data set and fit an appropriate GAM. Note that the variable *MeasurementDate* is indeed a Date. However, this variable was read in as a factor. To be able to produce a meaningful graph, we must declare this variable to be a date (with the `as.Date()` function).

```
## 1) get data
d.pancia <- read.table("../../Datasets/Crescita.txt",
                       header = TRUE)
##
## 2) declare Dates as such
d.pancia$MeasurementDate_AsDate <- as.Date(
  d.pancia$MeasurementDate,
  format = "%d-%m-%Y")
## check
head(d.pancia)
```

```
  MeasurementDate Circonferenza Week MeasurementDate_AsDate
1      15-03-2015            76   12             2015-03-15
2      20-03-2015            79   13             2015-03-20
3      29-03-2015            83   15             2015-03-29
4      05-04-2015            81   16             2015-04-05
5      15-04-2015            85   17             2015-04-15
6      19-04-2015            83   18             2015-04-19
```

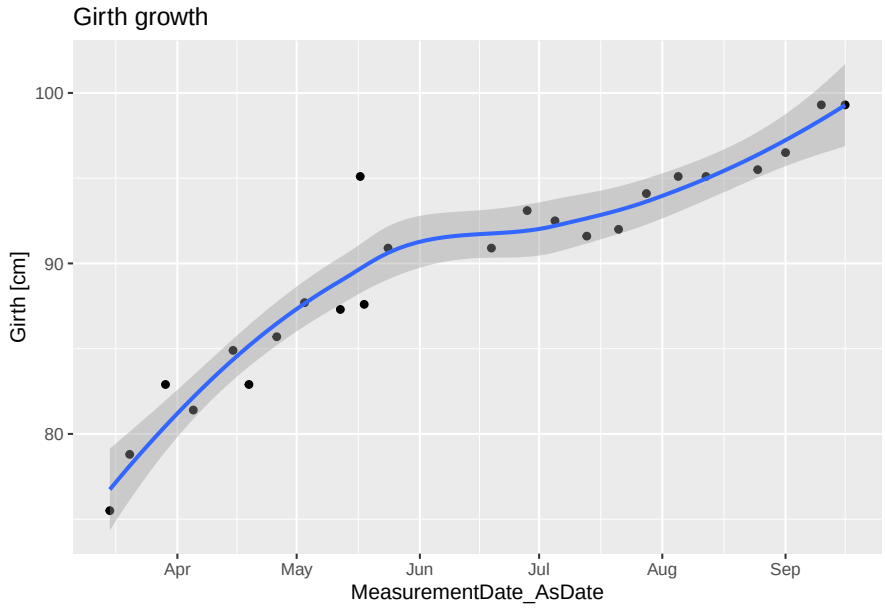
[4]Remember that regression and smoothing splines are only rarely used in practice.

```
str(d.pancia)
```

```
'data.frame':   24 obs. of  4 variables:
 $ MeasurementDate       : chr  "15-03-2015" "20-03-2015" "29-03-2015" "05-04-2015" ...
 $ Circonferenza         : num  75.5 78.8 82.9 81.4 84.9 82.9 85.7 87.7 87.3 87.6 ...
 $ Week                  : int  12 13 15 16 17 18 19 20 21 22 ...
 $ MeasurementDate_AsDate: Date, format: "2015-03-15" "2015-03-20" ...
```

Display the data.

```
gg.pancia <- ggplot(data = d.pancia,
       mapping = aes(y = Circonferenza,
                     x = MeasurementDate_AsDate)) +
  geom_point() +
  geom_smooth() +
  ylab("Girth [cm]") +
  ggtitle("Girth growth")
gg.pancia
```



The girth growth is clearly non-linear. Note that `ggplot` objects can be saved for later use (e.g. reuse them by adding layers).

Fit the GAM. Note that the `gam()` function does not work with `Date` objects. Therefore, you can use *Week* as a predictor or declare the *Date* object to be a numeric predictor (code is hidden; see the corresponding **R**-code for the solution).

Display the estimated smooth term (again: Code is hidden; see the corresponding **R**-code for the solution).

Alternatively, you may choose another data set that contains at least one continuous variable and fit a GAM.

8 Appendix

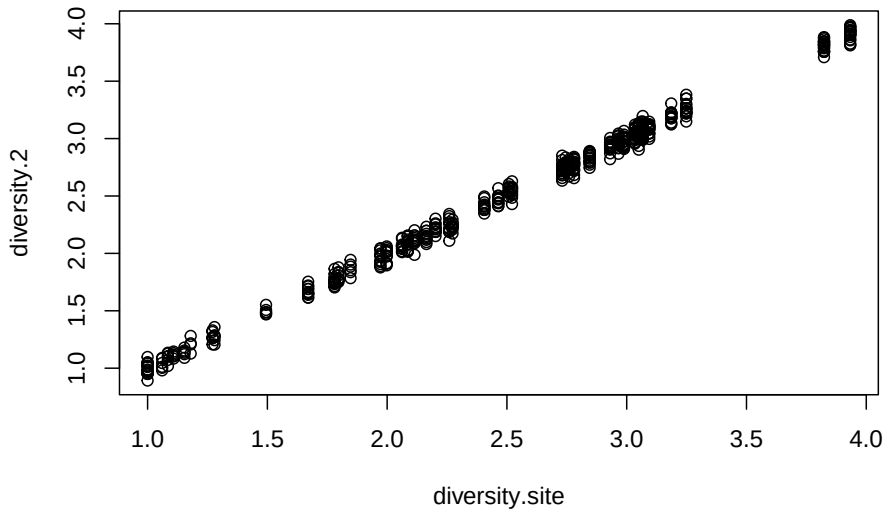
8.1 Collinearity issues

Sometimes we may want to include in a model predictors that are have very similar values. For example, in this case we may want to include a predictor that quantifies diversity in a different way than the one used so far (i.e. *diversity.site*). Indeed, there are a wide range of indexes that quantifies biodiversity. Not knowing which one to pick, we may be tempted to include several of them in the model and then choose the one that "has the lowest p-value".

Question: *There are two major problems with adding several indexes of biodiversity into a model. Can you explain them?*

Let's simulate the situation where we would add an additional biodiversity predictor that is pretty much the same as *diversity.site*.

```
set.seed(12)
d.trees$diversity.2 <- d.trees$diversity.site +
  rnorm(n = nrow(d.trees), sd = 0.05)
##
plot(diversity.2 ~ diversity.site, data = d.trees)
```



As the graph indicates, these two predictors are highly correlated. Let's now look at the effects of having two correlated predictors in a model.

We first fit a very simple model that only contains *diversity.site*, species and age.

```
lm.trees.4 <- lm( growth.rate ~ species + age + diversity.site,
                  data = d.trees)
summary(lm.trees.4)
```

```
Call:
lm(formula = growth.rate ~ species + age + diversity.site, data = d.trees)
```

```
Residuals:
    Min      1Q  Median      3Q     Max
-1.0548 -0.1504  0.0295  0.1770  0.6637

Coefficients:
                Estimate Std. Error t value Pr(>|t|)
(Intercept)     1.14e+00   6.12e-02   18.67  < 2e-16 ***
speciesLarch   -2.98e-01   3.09e-02   -9.62  < 2e-16 ***
speciesOak     -2.00e-01   3.04e-02   -6.58  1.1e-10 ***
speciesSpruce  -8.86e-02   3.07e-02   -2.89   0.0040 **
age            -4.13e-05   5.30e-04   -0.08   0.9380
diversity.site  4.40e-02   1.51e-02    2.92   0.0036 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.26 on 551 degrees of freedom
Multiple R-squared:  0.177, Adjusted R-squared:  0.17
F-statistic: 23.7 on 5 and 551 DF,  p-value: <2e-16
```

There is strong evidence that *diversity.site* plays a role (p-value < 0.01).

Let's now include the other "diversity predictor" and check the results again.

```
lm.trees.5 <- update(lm.trees.4, . ~ . + diversity.2)
summary(lm.trees.5)
```

```
Call:
lm(formula = growth.rate ~ species + age + diversity.site + diversity.2,
    data = d.trees)

Residuals:
    Min      1Q  Median      3Q     Max
-1.0547 -0.1502  0.0295  0.1770  0.6637

Coefficients:
                Estimate Std. Error t value Pr(>|t|)
(Intercept)     1.14e+00   6.13e-02   18.65  < 2e-16 ***
speciesLarch   -2.98e-01   3.10e-02   -9.60  < 2e-16 ***
speciesOak     -2.00e-01   3.04e-02   -6.57  1.2e-10 ***
speciesSpruce  -8.86e-02   3.08e-02   -2.88   0.0041 **
age            -4.12e-05   5.31e-04   -0.08   0.9383
diversity.site  4.54e-02   2.30e-01    0.20   0.8434
diversity.2    -1.44e-03   2.30e-01   -0.01   0.9950
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.26 on 550 degrees of freedom
Multiple R-squared:  0.177, Adjusted R-squared:  0.168
F-statistic: 19.7 on 6 and 550 DF,  p-value: <2e-16
```

Surprise! None of the "diversity" predictors seems to be important! Their p-values are, indeed, very large.

This misleading result is the consequence of having collinear predictors in the model. By carefully looking at the estimated coefficients for the predictor *diversity.site*, you will notice that they do not differ a lot between the two models.

```
coef(lm.trees.4)["diversity.site"]
```

```
diversity.site
         0.044
```

```
##
coef(lm.trees.5)["diversity.site"]
```

```
diversity.site
         0.045
```

What really changes between the two models are the standard errors (or in other words, the precision of these estimates).

```
summary(lm.trees.4)$coefficients["diversity.site", ]
```

```
  Estimate Std. Error    t value   Pr(>|t|)
    0.0440     0.0151     2.9212     0.0036
```

```
##
summary(lm.trees.5)$coefficients["diversity.site", ]
```

```
  Estimate Std. Error    t value   Pr(>|t|)
     0.045      0.230      0.198      0.843
```

The standard errors for the *diversity.site* coefficient is about 15 times larger in the second model! Therefore, the associated p-values are much larger even though the estimated effects stays almost the same.

So, whenever two or more highly correlated predictors are included in a model, it is very likely that none of them will appear to be significant. By concluding that they don't play any role, we would make a serious mistake in terms of results interpretation.

8.2 More on collinearity (**)

Note that above we looked at a very simple case where two variables were highly correlated. However, there are other situations that can lead to collinearity issues. If one variable is correlated to a linear combination of other variables, then there will be collinearity issues too. This is the reason why we look at the VIF factor (see next section) rather than looking at all pairwise correlations between predictors.

8.3 Measuring collinearity

It is sometimes difficult to know a-priori whether a model is affected by problems of collinearity. Fortunately, there are methods to estimate collinearity. The `vif()` function in the add-on *{car}* package offers a very convenient way to estimate collinearity.

```
library(car)
```

```
Loading required package: carData
```

```
vif(lm.trees.4)
```

```
                GVIF Df GVIF^(1/(2*Df))
species         1.0  3               1
age             1.1  1               1
diversity.site  1.1  1               1
```

The GVIF (Generalised Variance Inflation Factor) value tell us how serious is the issue of collinearity. A value above five is usually regarded as "if possible, remove that variable". A value above 10 means that the collinearity is too high and some action must be undertaken.

In the `lm.trees.4` model there does not seem to be any collinearity problem (i.e. GVIFs are below 5).

Let's compute the VIF for the model affected by collinearity.

```
vif(lm.trees.5)
```

```
                 GVIF Df GVIF^(1/(2*Df))
species          1.0  3               1
age              1.1  1               1
diversity.site 246.1  1              16
diversity.2    245.8  1              16
```

As expected, GVIF is way larger than 10 and some action must be undertaken. In this case, we should simply choose one of the two diversity predictors and drop the other from the model.

8.4 Predictive models with collinear predictors (*)

As mentioned above, a model with collinear predictors loose its interpretability in terms of the regression coefficients and p-values. However, if the scope of the model is purely to make predictions, there is no problem about including collinear predictors. Note, however that for prediction purposes, Machine Learning methods are often more suitable choices (due to higher performance).

8.5 Further capabilities of `gam()` (**)

8.5.1 Interactions with smooth terms (**)

In a previous Lab we have seen that some variables appeared to interact with species. We may want to inspect whether a different smooth term for *density.site* is needed for each species. This is possible within the `s()` function by specifying the "by" argument.

```
gam.trees.3 <- gam(growth.rate ~ species +
                     s(density.site, by = species),
                   data = d.trees)
summary(gam.trees.3)
```

8.5.2 2-dimensional smooth terms (**)

Smooth terms can be fitted on two-dimensions. A classical example would be to model the spatial effect on the response variable with a two-dimensional smooth term. This may represent, for example, soil fertility.

```
gam.trees.4 <- gam(growth.rate ~ species +
                     s(size) + s(X, Y), ## X and Y are coordinates
                   data = d.trees)
```

8.5.3 Model diagnostics for GAMs

GAMs make also assumptions about the distribution of the data. When using the `gam()` function to fit models one can specify the data distribution via the "family" argument. By default this argument is set to "gaussian" (i.e. normal distribution). Type `?family.mgcv` to see which families are implemented.

The very first line of a printed GAM (or its summary output) show what distribution is assumed.

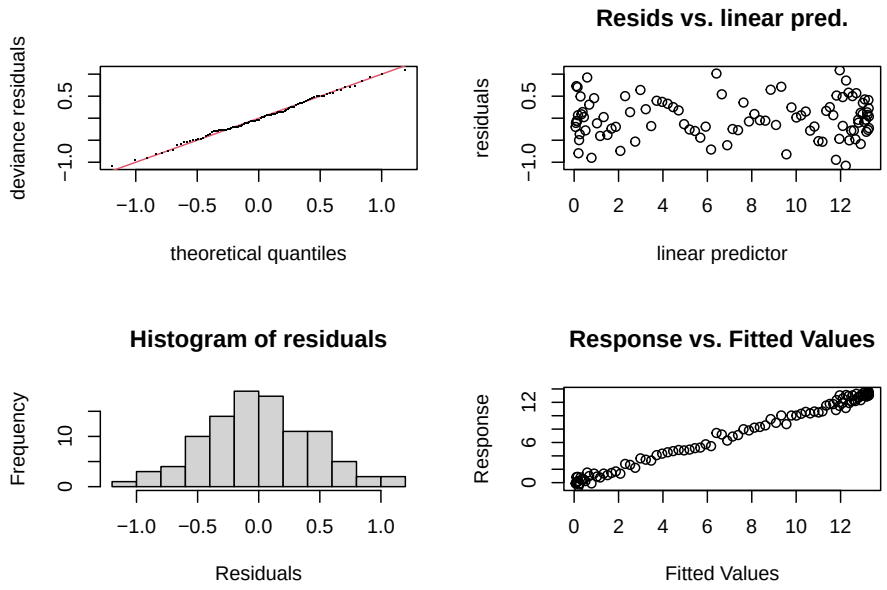
```
gam.1
```

```
Family: gaussian
Link function: identity

Formula:
yy ~ s(x1)

Estimated degrees of freedom:
7.11  total = 8.1

GCV score: 0.23
```

```
## or
# summary(gam.1)
```

We can (and should) test whether the model assumptions are fullfilled (the same ones as for a Linear model when assuming normality). To do so, we can either recreate all relevant graphs by ourselves or use the `gam.check()` function.

```
gam.check(gam.1)
```




```
Method: GCV   Optimizer: magic
Smoothing parameter selection converged after 8 iterations.
The RMS GCV score gradient at convergence was 7.9e-07 .
The Hessian was positive definite.
Model rank =  10 / 10

Basis dimension (k) checking results. Low p-value (k-index<1) may
indicate that k is too low, especially if edf is close to k'.

        k'  edf k-index p-value
s(x1) 9.00 7.11    1.08    0.75
```

8.6 The `I()` function

In this lab we fitted a model where the effect of *density.site* was allowed to have a linear term as well as a quadratic term. To specify a quadratic effect we used the "^2" symbol within the `I()` function.

```
formula(lm.trees.2)
```

```
growth.rate ~ species + age + diversity.site + density.site +
    I(density.site^2)
```

```
# summary(lm.trees.2)
```

Note that it is recommended to use the `I()` function here to make clear that we really want to square the predictor rather than having a two-fold interaction. Indeed, the "^2" symbol can also mean two-interactions in **R** formulae:

```
lm.two.fold <- lm(growth.rate ~ (species + age + diversity.site + density.site + density.site)^2,
                  data = d.trees)
summary(lm.two.fold)
```

```
Call:
lm(formula = growth.rate ~ (species + age + diversity.site +
    density.site + density.site)^2, data = d.trees)

Residuals:
    Min      1Q  Median      3Q     Max
-1.0350 -0.1541  0.0302  0.1736  0.6106

Coefficients:
                             Estimate Std. Error t value Pr(>|t|)
(Intercept)                  7.70e-01   3.36e-01    2.29   0.0223 *
speciesLarch                -6.11e-01   2.06e-01   -2.97   0.0032 **
speciesOak                  -6.44e-01   2.08e-01   -3.10   0.0020 **
speciesSpruce               -1.13e-01   2.67e-01   -0.42   0.6722
age                          1.74e-03   3.63e-03    0.48   0.6315
diversity.site               2.45e-01   7.70e-02    3.19   0.0015 **
density.site                 4.17e-03   7.64e-03    0.55   0.5851
speciesLarch:age            -3.72e-03   1.62e-03   -2.29   0.0223 *
speciesOak:age               3.64e-03   1.68e-03    2.16   0.0309 *
speciesSpruce:age           -2.48e-03   1.80e-03   -1.37   0.1698
speciesLarch:diversity.site  2.90e-02   4.31e-02    0.67   0.5008
speciesOak:diversity.site    9.76e-02   4.53e-02    2.16   0.0315 *
speciesSpruce:diversity.site 2.55e-03   5.20e-02    0.05   0.9608
speciesLarch:density.site    1.16e-02   3.62e-03    3.20   0.0014 **
speciesOak:density.site     -1.76e-03   3.84e-03   -0.46   0.6478
speciesSpruce:density.site   3.86e-03   4.06e-03    0.95   0.3415
age:diversity.site          -1.35e-03   8.37e-04   -1.62   0.1064
age:density.site             4.83e-05   6.62e-05    0.73   0.4663
diversity.site:density.site -3.26e-03   1.98e-03   -1.64   0.1008
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.25 on 538 degrees of freedom
Multiple R-squared:  0.252, Adjusted R-squared:  0.227
F-statistic: 10.1 on 18 and 538 DF,  p-value: <2e-16
```

The output above shows that the "^2" symbol can also be used to specify two fold interaction.

9 Session Information

```
sessionInfo()
```

```
R version 4.5.1 (2025-06-13)
Platform: aarch64-apple-darwin20
Running under: macOS Sequoia 15.6.1

Matrix products: default
BLAS:   /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRblas.0.dylib
LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;  LAPACK versi

locale:
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8

time zone: Europe/Zurich
tzcode source: internal

attached base packages:
[1] splines   stats     graphics  grDevices utils     datasets  methods
[8] base

other attached packages:
[1] car_3.1-3     carData_3.0-5 mgcv_1.9-3    nlme_3.1-168  gridExtra_2.3
[6] ggplot2_3.5.2 knitr_1.50

loaded via a namespace (and not attached):
 [1] Matrix_1.7-3       gtable_0.3.6       dplyr_1.1.4        compiler_4.5.1
 [5] tidyselect_1.2.1   tinytex_0.57       scales_1.4.0       yaml_2.3.10
 [9] fastmap_1.2.0      lattice_0.22-7     R6_2.6.1           labeling_0.4.3
[13] generics_0.1.4     Formula_1.2-5      tibble_3.3.0       pillar_1.11.0
[17] RColorBrewer_1.1-3 rlang_1.1.6        xfun_0.52          cli_3.6.5
[21] withr_3.0.2        magrittr_2.0.3     digest_0.6.37      grid_4.5.1
[25] rstudioapi_0.17.1  lifecycle_1.0.4    vctrs_0.6.5        evaluate_1.0.4
[29] glue_1.8.0         farver_2.1.2       abind_1.4-8        rmarkdown_2.29
[33] tools_4.5.1        pkgconfig_2.0.3    htmltools_0.5.8.1
```